# 算法常见优化技巧

## Tips：一些细节的实现可能会大幅提高效率

**Rosaya**

Oasis

# 目录


1. 前缀和 ..... 2
   1.1 一维前缀和 ..... 3
   1.2 二维前缀和 ..... 8
   1.3 高维前缀和 ..... 11
   1.4 习题 ..... 12
2. 差分 ..... 13
   2.1 一维差分 ..... 14
   2.2 高维差分 ..... 18
   2.3 习题 ..... 21
3. 离散化 ..... 22
   3.1 简介 ..... 23
   3.2 习题 ..... 26
4. 二分 ..... 27
   4.1 引入 ..... 28
   4.2 二分查找 ..... 29
   4.3 二分答案 ..... 32
   4.4 习题 ..... 35
5. 双指针 ..... 36
   5.1 分析 ..... 37
   5.2 例题 ..... 38
   5.3 习题 ..... 40

# 目录


1. 前缀和 ..... 2
   1.1 一维前缀和 ..... 3
   1.2 二维前缀和 ..... 8
   1.3 高维前缀和 ..... 11
   1.4 习题 ..... 12
2. 差分 ..... 13
   2.1 一维差分 ..... 14
   2.2 高维差分 ..... 18
   2.3 习题 ..... 21
3. 离散化 ..... 22
   3.1 简介 ..... 23
   3.2 习题 ..... 26
4. 二分 ..... 27
   4.1 引入 ..... 28
   4.2 二分查找 ..... 29
   4.3 二分答案 ..... 32
   4.4 习题 ..... 35
5. 双指针 ..... 36
   5.1 分析 ..... 37
   5.2 例题 ..... 38
   5.3 习题 ..... 40

## 1.1 一维前缀和

## 1.1 一维前缀和

对长度为 $n$ 的数组 $\{a_i\}$，我们定义前缀和为：

$$b_i = \sum_{i=1}^{n} a_i$$

这样我们还能找到一个递推式：

$$b_i = b_{i-1} + a_i$$

## 1.1 一维前缀和

对长度为 $n$ 的数组 $\{a_i\}$，我们定义前缀和为：

$$b_i = \sum_{i=1}^{n} a_i$$

这样我们还能找到一个递推式：

$$b_i = b_{i-1} + a_i$$

这样当我们想求 $i$ 到 $j$ 的和时，我们只需要：

$$\sum_{i=j}^{k} a_i = b_k - b_{j-1}$$

可以看到我们预处理的复杂度为 $O(n)$，而查询**区间和**只需要 $O(1)$ 的时间。

## 1.1 一维前缀和

## 1.1 一维前缀和

其实不仅是加法，前缀和也可以用来处理其他的操作，比如说：

- 区间乘法
- 区间异或
- ...

**逆运算**是可以用前缀和处理区间问题的必要条件，但即使不满足逆运算，前缀和也可以用来处理一些问题，比如说：

- 前缀最值
- 前缀最大公因数/最小公倍数
- ...

## 1.1 一维前缀和

## 1.1 一维前缀和

### P1115 最大子段和

给出一个长度为 $n$ 的序列 $a$，选出其中连续且非空的一段使得这段和最大。

$1 \le n \le 2 \times 10^5$，$-10^4 \le a_i \le 10^4$。

## 1.1 一维前缀和

### P1115 最大子段和

给出一个长度为 $n$ 的序列 $a$，选出其中连续且非空的一段使得这段和最大。

$1 \le n \le 2 \times 10^5$，$-10^4 \le a_i \le 10^4$。

考虑 $a_i$ 的前缀和 $b_i$，最大子段和为：

$$\max_{i=1}^{n}\left\{\max_{j=1}^{i}\{b_i - b_{j-1}\}\right\}$$

那我们可以考虑拆分贡献为：

$$\max_{i=1}^{n}\left\{b_i + \max_{j=1}^{i}\{-b_{j-1}\}\right\} = \max_{i=1}^{n}\left\{b_i - \min_{j=1}^{i}\{b_{j-1}\}\right\}$$

是前缀 min 形式，可以 $O(n)$ 解决。

## 1.1 一维前缀和

## 1.1 一维前缀和

### P2642 最大两段子段和

### P1121 环状最大两段子段和

给出一段长度为 $n$ 的**环状序列** $a$，即认为 $a_1$ 和 $a_n$ 是相邻的，选出其中连续不重叠且非空的两段使得这两段和最大。

$2 \le n \le 2 \times 10^5$，$-10^4 \le a_i \le 10^4$。

## 1.1 一维前缀和

### P2642 最大两段子段和

### P1121 环状最大两段子段和

给出一段长度为 $n$ 的**环状序列** $a$，即认为 $a_1$ 和 $a_n$ 是相邻的，选出其中连续不重叠且非空的两段使得这两段和最大。

$2 \le n \le 2 \times 10^5$，$-10^4 \le a_i \le 10^4$。

显然根据题意我们存在以下几种最终答案的情况：

- 选出两个长度均 $\ge 1$ 的区间。
- 选出三段区间，其中第一段包含 $a_1$，最后一段包含 $a_n$。

## 1.1 一维前缀和

## 1.1 一维前缀和

然后我们可以开始计算答案了，这里只计算第一类，第二类可以通过**总和**减去未选择的部分（最小两段子段和）来解决。

记 $f_i = b_i - \min\limits_{j=1}^{i}\{b_{j-1}\}, g_i = \max\limits_{j=i}^{n}\{b_j\} - b_{i-1}$，这样我们枚举两段的分界点 $i$ 使得左侧一段在 $[1, i]$ 中，右侧一段在 $[i+1, n]$ 中。

## 1.1 一维前缀和

然后我们可以开始计算答案了，这里只计算第一类，第二类可以通过**总和**减去未选择的部分（最小两段子段和）来解决。

记 $f_i = b_i - \min\limits_{j=1}^{i}\{b_{j-1}\}, g_i = \max\limits_{j=i}^{n}\{b_j\} - b_{i-1}$，这样我们枚举两段的分界点 $i$ 使得左侧一段在 $[1, i]$ 中，右侧一段在 $[i+1, n]$ 中。

这样我们就可以得到答案为：

$$\max_{i=1}^{n+1}\left\{\max_{j=1}^{i}\{f_j\} + \max_{j=i+1}^{n}\{g_j\}\right\}$$

分别是前缀 max 和后缀 max 的形式，可以 $O(n)$ 解决。

## 1.2 二维前缀和

## 1.2 二维前缀和

当然对于二维数组，我们也可以定义：

$$b_{i,j} = \sum_{x=1}^{i} \sum_{y=1}^{j} a_{i,j}$$

为二维前缀和。

**二维前缀和的递推式？**

## 1.2 二维前缀和

当然对于二维数组，我们也可以定义：

$$b_{i,j} = \sum_{x=1}^{i} \sum_{y=1}^{j} a_{i,j}$$

为二维前缀和。

### 二维前缀和的递推式？

$$b_{i,j} = b_{i-1,j} + b_{i,j-1} - b_{i-1,j-1} + a_{i,j}$$

此时我们可以用 $O(n^2)$ 的时间预处理出前缀和，查询 $O(1)$。

$$\sum_{x=x_0}^{x_1} \sum_{y=y_0}^{y_1} a_{x,y} = b_{x_1,y_1} - b_{x_0-1,y_1} - b_{x_1,y_0-1} + b_{x_0-1,y_0-1}$$

## 1.2 二维前缀和

## 1.2 二维前缀和

### P1719 最大加权矩形

给定一个 $n \times n$ 的矩阵 $a$，每个元素 $a_{\{i,j\}}$ 表示该点的权值，求出一个矩形区域的权值和最大。

$1 \leq n \leq 120$。

## 1.2 二维前缀和

### P1719 最大加权矩形

给定一个 $n \times n$ 的矩阵 $a$，每个元素 $a_{\{i,j\}}$ 表示该点的权值，求出一个矩形区域的权值和最大。

$1 \leq n \leq 120$。

$1 \leq n \leq 500$。

## 1.2 二维前缀和

### P1719 最大加权矩形

给定一个 $n \times n$ 的矩阵 $a$，每个元素 $a_{\{i,j\}}$ 表示该点的权值，求出一个矩形区域的权值和最大。

$1 \le n \le 120$。

$1 \le n \le 500$。

直接枚举是 $O(n^4)$ 的算法，还有没有更好的做法？

## 1.2 二维前缀和

### P1719 最大加权矩形

给定一个 $n \times n$ 的矩阵 $a$，每个元素 $a_{\{i,j\}}$ 表示该点的权值，求出一个矩形区域的权值和最大。

$1 \le n \le 120$。

$1 \le n \le 500$。

直接枚举是 $O(n^4)$ 的算法，还有没有更好的做法？

其实是有的。我们考虑枚举矩形的上下边界 $i, j$，然后我们可以将矩形的每一列都**拍成一个数**。

这样我们相当于进行一维的最大子段和，总复杂度 $O(n^3)$。

## 1.2 二维前缀和

## 1.2 二维前缀和

### P3941 入阵曲

给定一个 $n \times m$ 的矩阵 $a$，求有多少子矩形的数字之和是 $k$ 的倍数。

$1 \le n, m \le 400$，$1 \le k \le 10^6$。

## 1.2 二维前缀和

### P3941 入阵曲

给定一个 $n \times m$ 的矩阵 $a$，求有多少子矩形的数字之和是 $k$ 的倍数。

$1 \le n, m \le 400$，$1 \le k \le 10^6$。

还是像最大子矩形一样，我们枚举上下边界 $i, j$，然后我们可以将矩形的每一列都**拍成一个数**。

这样我们相当于是一维有多少个区间的和是 $k$ 的倍数。

对于 $i < j$，若 $b_{i-1} \equiv b_j \pmod k$，则 $[i, j]$ 是和为 $k$ 的倍数的矩形，使用数组统计即可。

复杂度 $O(n^3)$。

## 1.3 高维前缀和

## 1.3 高维前缀和

这部分是**拓展内容**。

考虑二维向三维，我们发现递推式会变为：

$$
\begin{aligned}
b_{i,j,k} = b_{i-1,j,k} + b_{i,j-1,k} + b_{i,j,k-1} - b_{i-1,j-1,k} \\
-b_{i-1,j,k-1} - b_{i,j-1,k-1} + b_{i-1,j-1,k-1}
\end{aligned}
$$

再考虑四维.....

我们发现 $k$ 维的复杂度变成了 $O(2^k)$，这显然是不能接受的。

## 1.3 高维前缀和

这部分是**拓展内容**。

考虑二维向三维，我们发现递推式会变为：

$$\begin{aligned} b_{i,j,k} = b_{i-1,j,k} + b_{i,j-1,k} + b_{i,j,k-1} - b_{i-1,j-1,k} \\ -b_{i-1,j,k-1} - b_{i,j-1,k-1} + b_{i-1,j-1,k-1} \end{aligned}$$

再考虑四维.....

我们发现 $k$ 维的复杂度变成了 $O(2^k)$，这显然是不能接受的。

事实上，我们发现对每一维都做一次一维前缀和即可，这样复杂度仅为 $O(k)$。

这个算法**完全是对的**！等我们了解完差分之后，我们会用它来解决 FWT 卷积。

## 1.3 高维前缀和

这部分是**拓展内容**。

考虑二维向三维，我们发现递推式会变为：

$$\begin{aligned} b_{i,j,k} = b_{i-1,j,k} + b_{i,j-1,k} + b_{i,j,k-1} - b_{i-1,j-1,k} \\ -b_{i-1,j,k-1} - b_{i,j-1,k-1} + b_{i-1,j-1,k-1} \end{aligned}$$

再考虑四维.....

我们发现 $k$ 维的复杂度变成了 $O(2^k)$，这显然是不能接受的。

事实上，我们发现对每一维都做一次一维前缀和即可，这样复杂度仅为 $O(k)$。

这个算法**完全是对的**！等我们了解完差分之后，我们会用它来解决 FWT 卷积。

事实上，如果我们用质因子分解来得到这个**高维向量**，我们就得到了 **Dirichlet 卷积**。

## 1.4 习题

- **P2004 领地选择**
- **P8865 [NOIP2022] 种花**
- **P2671 [NOIP 2015 普及组] 求和**
- **P2119 [NOIP 2016 普及组] 魔法阵**

# 目录


1. 前缀和 ..... 2
   1.1 一维前缀和 ..... 3
   1.2 二维前缀和 ..... 8
   1.3 高维前缀和 ..... 11
   1.4 习题 ..... 12
2. 差分 ..... 13
   2.1 一维差分 ..... 14
   2.2 高维差分 ..... 18
   2.3 习题 ..... 21
3. 离散化 ..... 22
   3.1 简介 ..... 23
   3.2 习题 ..... 26
4. 二分 ..... 27
   4.1 引入 ..... 28
   4.2 二分查找 ..... 29
   4.3 二分答案 ..... 32
   4.4 习题 ..... 35
5. 双指针 ..... 36
   5.1 分析 ..... 37
   5.2 例题 ..... 38
   5.3 习题 ..... 40

## 2.1 一维差分

## 2.1 一维差分

对长度为 $n$ 的数组 $\{a_i\}$，我们定义差分为：

$$b_i = a_i - a_{i-1}$$

这里认为 $a_0 = 0$，复杂度 $O(n)$。

我们可以看到，差分其实就是前缀和的**逆运算**。

因为 $\sum_{j=1}^{i} b_j = a_i$。

一般用于处理**区间操作问题**。

例：区间 $[l, r]$ 加上 $k$，可以认为是 $b_l$ 加 $k$ 而 $b_{r+1}$ 减 $k$，最后再做前缀和还原数组。

## 2.1 一维差分

## 2.1 一维差分

### P2367 语文成绩

语文老师总是写错成绩，所以当她修改成绩的时候，她会给 $[l, r]$ 区间内的同学统一加 $k$ 分。在加完所有分后，她想知道最低分是多少。

$1 \le n \le 5 \times 10^6$，$p \le n$，$a_i \le 100$，$z \le 100$。

## 2.1 一维差分

### P2367 语文成绩

语文老师总是写错成绩，所以当她修改成绩的时候，她会给 $[l, r]$ 区间内的同学统一加 $k$ 分。在加完所有分后，她想知道最低分是多少。

$1 \le n \le 5 \times 10^6$，$p \le n$，$a_i \le 100$，$z \le 100$。

与前面说的相同，处理完差分数组再前缀和还原即可。

复杂度 $O(n)$。

## 2.1 一维差分

## 2.1 一维差分

### P4552 [Poetize6] IncDec Sequence

给定一个长度为 $n$ 的数列 $a_1, a_2, ..., a_n$，每次可以选择一个区间 $[l, r]$，使这个区间内的数都加 1 或者都减 1。

请问至少需要多少次操作才能使数列中的所有数都一样，并求出在**保证最少次数**的前提下，最终得到的数列有多少种。

$1 \le n \le 10^5$，$0 \le a_i \le 2^{31}$。

## 2.1 一维差分

## 2.1 一维差分

先考虑最少步数，我们发现每次可以把一个 $b_i$ 加一，一个 $b_j$ 减一，由于区间**可以加，可以减**，故我们对 $i, j$ 的位置关系没有限制。**注：**如果**只能加一**，那么必须 $i < j$。减一同理。

那么显然每个正的差分都要和负的差分**匹配**，一次操作最多消掉一正一负，故答案为：

$$\max\left\{\sum_{i=2}^{n}\max\{0, a_i - a_{i-1}\}, \sum_{i=2}^{n}\max\{0, a_{i-1} - a_i\}\right\}$$

。

## 2.1 一维差分

先考虑最少步数，我们发现每次可以把一个 $b_i$ 加一，一个 $b_j$ 减一，由于区间**可以加，可以减**，故我们对 $i, j$ 的位置关系没有限制。**注：**如果**只能加一**，那么必须 $i < j$。减一同理。

那么显然每个正的差分都要和负的差分**匹配**，一次操作最多消掉一正一负，故答案为：

$$\max\left\{\sum_{i=2}^{n}\max\{0, a_i - a_{i-1}\}, \sum_{i=2}^{n}\max\{0, a_{i-1} - a_i\}\right\}$$

。

当然你可能在想**正负差分不应该刚好抵消吗？**

所以我们发现如果你把序列看成一个环，也就是 $b_1 = a_1 - a_n$，那么此时肯定平衡。我们对于 $b_1$ 的消除**可以操作前缀、可以操作后缀**。那么也就是说最终得到的答案在 $a_1 \sim a_n$ 都有可能，答案为 $|a_1 - a_n|$，总复杂度 $O(n)$。

## 2.2 高维差分

## 2.2 高维差分

跟高维前缀和相同，只需要**一维一维**分开处理即可做到 $O(k)$ 计算一个值的复杂度。

在 FWT 之前，我们先来看个简单的问题：

## 2.2 高维差分

跟高维前缀和相同，只需要**一维一维**分开处理即可做到 $O(k)$ 计算一个值的复杂度。

在 FWT 之前，我们先来看个简单的问题：

给定长度为 $n$ 的数组 $A, B$，定义 $C_i = \sum\limits_{\max\{j,k\}=i} A_j B_k$。

求 $C$。

$1 \le n \le 5 \times 10^6$。

## 2.2 高维差分

跟高维前缀和相同，只需要**一维一维**分开处理即可做到 $O(k)$ 计算一个值的复杂度。

在 FWT 之前，我们先来看个简单的问题：

给定长度为 $n$ 的数组 $A, B$，定义 $C_i = \sum\limits_{\max\{j,k\}=i} A_j B_k$。

求 $C$。

$1 \le n \le 5 \times 10^6$。

当然可以前缀和之后枚举，不过我们想想有没有**更简单的做法**。

## 2.2 高维差分

跟高维前缀和相同，只需要**一维一维**分开处理即可做到 $O(k)$ 计算一个值的复杂度。

在 FWT 之前，我们先来看个简单的问题：

给定长度为 $n$ 的数组 $A, B$，定义 $C_i = \sum_{\max\{j,k\}=i} A_j B_k$。

求 $C$。

$1 \le n \le 5 \times 10^6$。

当然可以前缀和之后枚举，不过我们想想有没有**更简单的做法**。

$A, B$ 做前缀和，$C_i = A_i B_i$，$C$ 再差分即可得到答案，复杂度 $O(n)$。为什么？

## 2.2 高维差分

跟高维前缀和相同，只需要**一维一维**分开处理即可做到 $O(k)$ 计算一个值的复杂度。

在 FWT 之前，我们先来看个简单的问题：

给定长度为 $n$ 的数组 $A, B$，定义 $C_i = \sum_{\max\{j,k\}=i} A_j B_k$。

求 $C$。

$1 \le n \le 5 \times 10^6$。

当然可以前缀和之后枚举，不过我们想想有没有**更简单的做法**。

$A, B$ 做前缀和，$C_i = A_i B_i$，$C$ 再差分即可得到答案，复杂度 $O(n)$。为什么？

其实这个东西是可以扔到**高维**上的。

## 2.2 高维差分

## 2.2 高维差分

### P4717【模板】快速莫比乌斯/沃尔什变换 (FMT/FWT)

给定长度为 $2^n$ 的两个序列 $A, B$，设

$$C_i = \sum_{j \oplus k = i} A_j B_k$$

分别当 $\oplus$ 是 or, and, xor 时求出 $C$。

$1 \le n \le 17$。

## 2.2 高维差分

### P4717【模板】快速莫比乌斯/沃尔什变换 (FMT/FWT)

给定长度为 $2^n$ 的两个序列 $A, B$，设

$$C_i = \sum_{j \oplus k = i} A_j B_k$$

分别当 $\oplus$ 是 or, and, xor 时求出 $C$。

$1 \le n \le 17$。

or 时为先高维前缀和，然后对应位相乘再差分。

and 时为先高维后缀和，然后对应位相乘再差分。

xor 时被称为 FWT，需要一些变换。

## 2.2 高维差分

## 2.2 高维差分

记 $x \otimes y = \text{popcount}(x \oplus y) \bmod 2$，也就是 $x \oplus y$ 的 1 的个数的奇偶性。

有性质 $(i \otimes j) \oplus (i \otimes k) = i \otimes (j \oplus k)$。

## 2.2 高维差分

记 $x \otimes y = \mathrm{popcount}(x \oplus y) \bmod 2$，也就是 $x \oplus y$ 的 1 的个数的奇偶性。

有性质 $(i \otimes j) \oplus (i \otimes k) = i \otimes (j \oplus k)$。

那么构造数组 $\mathrm{FWT}(a)_i = \sum\limits_{j \otimes i = 0} a_j - \sum\limits_{j \otimes i = 1} a_j$。

## 2.2 高维差分

记 $x \otimes y = \text{popcount}(x \oplus y) \bmod 2$，也就是 $x \oplus y$ 的 1 的个数的奇偶性。

有性质 $(i \otimes j) \oplus (i \otimes k) = i \otimes (j \oplus k)$。

那么构造数组 $\text{FWT}(a)_i = \sum\limits_{j \otimes i = 0} a_j - \sum\limits_{j \otimes i = 1} a_j$。

$$
\begin{aligned}
\text{FWT}[a]_i * \text{FWT}[b]_i &= \left( \sum_{j \otimes i = 0} a_j - \sum_{j \otimes i = 1} a_j \right) \left( \sum_{k \otimes i = 0} b_k - \sum_{k \otimes i = 1} b_k \right) \\
&= \sum_{(j \oplus k) \otimes i = 0} a_j b_k - \sum_{(j \oplus k) \otimes i = 1} a_j b_k \\
&= \text{FWT}[c]_i
\end{aligned}
$$

那么我们分治解决 $\text{FWT}[a] = \text{merge}(\text{FWT}[a_0] + \text{FWT}[a_1], \text{FWT}[a_0] - \text{FWT}[a_1])$，而逆变换有 $a = \text{merge}((a_0 + a_1)/2, (a_0 - a_1)/2)$，复杂度 $O(n2^n)$。

## 2.3 习题

- **P3406 海底高铁**
- **P8292 [省选联考 2022] 卡牌（*）**

# 目录


1. 前缀和 ..... 2
   1.1 一维前缀和 ..... 3
   1.2 二维前缀和 ..... 8
   1.3 高维前缀和 ..... 11
   1.4 习题 ..... 12
2. 差分 ..... 13
   2.1 一维差分 ..... 14
   2.2 高维差分 ..... 18
   2.3 习题 ..... 21
3. 离散化 ..... 22
   3.1 简介 ..... 23
   3.2 习题 ..... 26
4. 二分 ..... 27
   4.1 引入 ..... 28
   4.2 二分查找 ..... 29
   4.3 二分答案 ..... 32
   4.4 习题 ..... 35
5. 双指针 ..... 36
   5.1 分析 ..... 37
   5.2 例题 ..... 38
   5.3 习题 ..... 40

## 3.1 简介

## 3.1 简介

离散化是一种数据处理的技巧，本质上可以看成是一种**哈希**，其保证数据在哈希以后仍然保持原来的**全/偏序**关系。

$$f : \mathbb{F} \to \mathbb{N}^*$$

其中 $\mathbb{F}$ 是原数据，$\mathbb{N}^*$ 是离散化后的数据，$\mathbb{F}$ 可以是**大整数，小数，分数**等等存在**全/偏序**关系的数。

一般来讲我们会将 $n$ 个数离散化为 $n$ 个整数 $1, 2, \ldots, n$。

根据相同的数是否要离散化为不同的数，我们用两种离散化方法。

## 3.1 简介

## 3.1 简介

```
DISCRETIZING(n, a[1...n]):
b[1...n] ← a[1...n]
SORT(n, b[1...n])
for i from 1 to n:
    a[i] ← LOWER_BOUND(b, a[i])
return a[1...n]
```

## 3.1 简介

```
DISCRETIZING(n, a[1...n]):
b[1...n] ← a[1...n]
SORT(n, b[1...n])
for i from 1 to n:
    a[i] ← LOWER_BOUND(b, a[i])
return a[1...n]
```

```
DISCRETIZING(n, a[1...n]):
b[1...n] ← a[1...n] × [1...n]
SORT(n, b[1...n])
for i from 1 to n:
    a[i] ← LOWER_BOUND(b, (a[i], i))
return a[1...n]
```

## 3.1 简介

## 3.1 简介

### P1496 火烧赤壁

给定 $n$ 条线段 $[a_i, b_i)$，求出线段并的长度。

$1 \le n \le 2 \times 10^4$，$2^{-31} \le a_i < b_i < 2^{31}$，答案在 int 范围内。

## 3.1 简介

### P1496 火烧赤壁

给定 $n$ 条线段 $[a_i, b_i)$，求出线段并的长度。

$1 \le n \le 2 \times 10^4$，$2^{-31} \le a_i < b_i < 2^{31}$，答案在 int 范围内。

离散化之后用差分和前缀和判断每一段是否被覆盖，复杂度 $O(n \log n)$。

代码比较好写，要快速实现。

## 3.1 简介

### P1496 火烧赤壁

给定 $n$ 条线段 $[a_i, b_i)$，求出线段并的长度。

$1 \le n \le 2 \times 10^4$，$2^{-31} \le a_i < b_i < 2^{31}$，答案在 int 范围内。

离散化之后用差分和前缀和判断每一段是否被覆盖，复杂度 $O(n \log n)$。

代码比较好写，要快速实现。

### P1884 [USACO12FEB] Overplanting S

求 $n$ 个矩形的并的面积。

$1 \le n \le 1000$，$-10^8 \le x_1, x_2, y_1, y_2 \le 10^8$。

## 3.2 习题

- **P1955 [NOI2015] 程序自动分析**
- **P1904 天际线**
- **P3029 [USACO11NOV] Cow Lineup S**

# 目录


1. 前缀和 ..... 2
   1.1 一维前缀和 ..... 3
   1.2 二维前缀和 ..... 8
   1.3 高维前缀和 ..... 11
   1.4 习题 ..... 12
2. 差分 ..... 13
   2.1 一维差分 ..... 14
   2.2 高维差分 ..... 18
   2.3 习题 ..... 21
3. 离散化 ..... 22
   3.1 简介 ..... 23
   3.2 习题 ..... 26
4. 二分 ..... 27
   4.1 引入 ..... 28
   4.2 二分查找 ..... 29
   4.3 二分答案 ..... 32
   4.4 习题 ..... 35
5. 双指针 ..... 36
   5.1 分析 ..... 37
   5.2 例题 ..... 38
   5.3 习题 ..... 40

## 4.1 引入

## 4.1 引入

我在心里想了一个 $1 \sim 15$ 之间的数，你每次可以猜一个数，我会告诉你大了还是小了。你在最坏情况下要最少问我几次能得到这个数？

## 4.1 引入

我在心里想了一个 $1 \sim 15$ 之间的数，你每次可以猜一个数，我会告诉你大了还是小了。你在最坏情况下要最少问我几次能得到这个数？

交互题和**自适应交互库**。

**自适应交互库**可能会在询问时更改答案，使询问次数最大。

## 4.1 引入

我在心里想了一个 $1 \sim 15$ 之间的数，你每次可以猜一个数，我会告诉你大了还是小了。你在最坏情况下要最少问我几次能得到这个数？

交互题和**自适应交互库**。

**自适应交互库**可能会在询问时更改答案，使询问次数最大。

lower_bound 的实现方式。

如何做到复杂度 $O(\log n)$？

## 4.2 二分查找

## 4.2 二分查找

lower_bound 的实现方式：

```
LOWER_BOUND(n, a[1...n], v):
p ← n + 1, l ← 1, r ← n
while l ≤ r:
    m ← (l + r)/2
    if a[m] ≥ v:
        p ← m, r ← m − 1   // check(m)
    else:
        l ← m + 1
return p
```

## 4.2 二分查找

lower_bound 的实现方式：

```
LOWER_BOUND(n, a[1...n], v):
p ← n + 1, l ← 1, r ← n
while l ≤ r:
    m ← (l + r)/2
    if a[m] ≥ v:
        p ← m, r ← m − 1   // check(m)
    else:
        l ← m + 1
return p
```

这也是常见的二分的实现方式，相较于 $l < r$ 的写法，**更便于理解**，**更不用动脑**。

复杂度为 $O(\log n \cdot check)$。

## 4.2 二分查找

## 4.2 二分查找

### P1020 [NOIP 1999 提高组] 导弹拦截

导弹拦截系统可以拦截一个高度单调不增的导弹序列，现给出长度为 $n$ 的导弹序列，求一个系统能拦截的最多导弹数量和拦截所有导弹需要的最少系统数量。

$1 \le n \le 10^5$，导弹高度不高于 $5 \times 10^4$。

## 4.2 二分查找

### P1020 [NOIP 1999 提高组] 导弹拦截

导弹拦截系统可以拦截一个高度单调不增的导弹序列，现给出长度为 $n$ 的导弹序列，求一个系统能拦截的最多导弹数量和拦截所有导弹需要的最少系统数量。

$1 \le n \le 10^5$，导弹高度不高于 $5 \times 10^4$。

第一问是**最长单调不增**子序列，我们考虑记 $f_i$ 表示以 $i$ 结尾的**最长单调不增**子序列长度。

我们考虑记 $g_i$ 为 $f_j = i$ 中 $a_j$ 的最大值，那么我们尝试动态维护这个东西。

## 4.2 二分查找

### P1020 [NOIP 1999 提高组] 导弹拦截

导弹拦截系统可以拦截一个高度单调不增的导弹序列，现给出长度为 $n$ 的导弹序列，求一个系统能拦截的最多导弹数量和拦截所有导弹需要的最少系统数量。

$1 \le n \le 10^5$，导弹高度不高于 $5 \times 10^4$。

第一问是**最长单调不增**子序列，我们考虑记 $f_i$ 表示以 $i$ 结尾的**最长单调不增**子序列长度。

我们考虑记 $g_i$ 为 $f_j = i$ 中 $a_j$ 的最大值，那么我们尝试动态维护这个东西。

首先需要一些性质：$g$ 是**单调不增**的。我们将在**归纳**中证明这件事。$i = 1$ 时显然，$i > 1$ 时在计算 $f_i$ 之前 $g$ 还是不降的，所以 $g_j \ge a_i$ 的 $j$ 是一个前缀，找到最大的 $j$，$f_i \leftarrow j + 1$，再 $g_{j+1} \leftarrow a_i$ 即可。

我们发现其实是一个类似 **upper_bound** 的东西，可以快速维护新的 $f_i$ 和更新 $g_{f_i}$，然后这一问就解决了，复杂度 $O(n \log n)$。

## 4.2 二分查找

## 4.2 二分查找

第二问我们又需要用到一个经典结论：**最小链覆盖 = 最长反链**，也就是 Dilworth 定理。这里反链就是**单调增**的。

那么只需要求**最长上升子序列**即可，复杂度 $O(n \log n)$。这正是 LIS。

在这道题中，如何证明该结论？

## 4.2 二分查找

第二问我们又需要用到一个经典结论：**最小链覆盖 = 最长反链**，也就是 Dilworth 定理。这里反链就是**单调增**的。

那么只需要求**最长上升子序列**即可，复杂度 $O(n \log n)$。这正是 LIS。

在这道题中，如何证明该结论？

证明**最小链覆盖 $\geq$ 最长反链**，再给出一组构造即可。

## 4.3 二分答案

## 4.3 二分答案

当然如果答案也是具有**可二分性**，也就是说可能的答案之间存在**全/偏序**关系，且有一个**分界点** $v$，满足 $\preceq v$ 的答案均**合法**，$\succ v$ 的答案均**不合法**，那么我们可以用二分来求解。

## 4.3 二分答案

当然如果答案也是具有**可二分性**，也就是说可能的答案之间存在**全/偏序**关系，且有一个**分界点** $v$，满足 $\preceq v$ 的答案均**合法**，$\succ v$ 的答案均**不合法**，那么我们可以用二分来求解。

```
GETANS():
p ← −1, l ← Min, r ← Max
while l ≤ r:
    m ← (l + r)/2
    if CHECK(m):
        p ← m, r ← m − 1
    else:
        l ← m + 1
return p
```

一般用于解决形如**最小值最大/最大值最小**的问题。

## 4.3 二分答案

## 4.3 二分答案

### CF830A Office Keys

在一条直线上有 $n$ 个人和 $k$ 个钥匙。每个人都想到达同样位于这条直线上的办公室。要做到这一点，他需要到达一个有钥匙的点上，拿到钥匙然后去办公室。

1. 一旦有人拿走某一把钥匙，任何人都不能拿走这把钥匙。

2. 人们每一秒移动一个单位距离。

需要求出所有人用钥匙到达办公室的最短时间。

$1 \le n \le 1000$，$n \le k \le 2000$，$1 \le p \le 10^9$。

## 4.3 二分答案

### CF830A Office Keys

在一条直线上有 $n$ 个人和 $k$ 个钥匙。每个人都想到达同样位于这条直线上的办公室。要做到这一点，他需要到达一个有钥匙的点上，拿到钥匙然后去办公室。

1. 一旦有人拿走某一把钥匙，任何人都不能拿走这把钥匙。

2. 人们每一秒移动一个单位距离。

需要求出所有人用钥匙到达办公室的最短时间。

$1 \le n \le 1000$，$n \le k \le 2000$，$1 \le p \le 10^9$。

二分答案，check 时可以 dp，记 $f_{i,j}$ 表示前 $i$ 个人拿 $j$ 个钥匙能否完成任务，显然 $f_{i,j} \leftarrow f_{i,j-1}$，若 $i$ 拿 $j$ 可行，则还有另一种转移 $f_{i,j} \leftarrow f_{i-1,j-1}$，复杂度 $O(nk \log V)$，其中 $V = 10^9$。当然也有一个贪心解法，感兴趣的同学可以自行探究。

## 4.3 二分答案

## 4.3 二分答案

当然二分答案还有一类重要的应用，称之为**分数规划**。

### 最大子段平均值

找到 $r-l+1 \geq k$ 的区间 $[l,r]$，使得 $\sum_{i=l}^{r} a_i/(r-l+1)$ 最大，**保留 2 位小数**。

$1 \leq n \leq 10^5$，$-10^5 \leq a_i \leq 10^5$。

## 4.3 二分答案

当然二分答案还有一类重要的应用，称之为**分数规划**。

### 最大子段平均值

找到 $r-l+1 \geq k$ 的区间 $[l,r]$，使得 $\sum_{i=l}^{r} a_i/(r-l+1)$ 最大，**保留 2 位小数**。

$1 \leq n \leq 10^5$，$-10^5 \leq a_i \leq 10^5$。

先考虑 $\left\lfloor \sum_{i=l}^{r} 1000a_i/(r-l+1) \right\rfloor$ 再四舍五入。

当我们二分到 $m$ 时，我们发现 $\left\lfloor \sum_{i=l}^{r} 1000a_i/(r-l+1) \right\rfloor \geq m$ 等同于 $\sum_{i=l}^{r} 1000a_i \geq m(r-l+1)$ 等同于 $\sum_{i=l}^{r}(1000a_i - m) \geq 0$，所以令 $b_i = 1000a_i - m$ 做**有区间限制的最大子段和**即可。

复杂度 $O(n \log V)$，其中 $V = 10^8$。

## 4.3 二分答案

当然二分答案还有一类重要的应用，称之为**分数规划**。

### 最大子段平均值

找到 $r-l+1 \geq k$ 的区间 $[l,r]$，使得 $\sum_{i=l}^{r} a_i/(r-l+1)$ 最大，**保留 2 位小数**。

$1 \leq n \leq 10^5$，$-10^5 \leq a_i \leq 10^5$。

先考虑 $\left\lfloor \sum_{i=l}^{r} 1000a_i/(r-l+1) \right\rfloor$ 再四舍五入。

当我们二分到 $m$ 时，我们发现 $\left\lfloor \sum_{i=l}^{r} 1000a_i/(r-l+1) \right\rfloor \geq m$ 等同于 $\sum_{i=l}^{r} 1000a_i \geq m(r-l+1)$ 等同于 $\sum_{i=l}^{r}(1000a_i - m) \geq 0$，所以令 $b_i = 1000a_i - m$ 做**有区间限制的最大子段和**即可。

复杂度 $O(n \log V)$，其中 $V = 10^8$。

求出**精确值** $p/q$，能不能做？

## 4.4 习题

- P1439【模板】最长公共子序列
- P1083 [NOIP 2012 提高组] 借教室
- P1462 通往奥格瑞玛的道路
- POJ2976 Dropping tests
- P4377 Talent Show
- P3199 [HNOI2009] 最小圈

# 目录


1. 前缀和 ..... 2
   1.1 一维前缀和 ..... 3
   1.2 二维前缀和 ..... 8
   1.3 高维前缀和 ..... 11
   1.4 习题 ..... 12
2. 差分 ..... 13
   2.1 一维差分 ..... 14
   2.2 高维差分 ..... 18
   2.3 习题 ..... 21
3. 离散化 ..... 22
   3.1 简介 ..... 23
   3.2 习题 ..... 26
4. 二分 ..... 27
   4.1 引入 ..... 28
   4.2 二分查找 ..... 29
   4.3 二分答案 ..... 32
   4.4 习题 ..... 35
5. 双指针 ..... 36
   5.1 分析 ..... 37
   5.2 例题 ..... 38
   5.3 习题 ..... 40

## 5.1 分析

## 5.1 分析

双指针是一种常见的**线性遍历**技巧，通常用于处理一些**区间问题**。

考虑固定左端点 $l$，我们可以用一个指针 $r$ 来**扩展**右端点。

如果我们记 $r_l$ 表示最小的 $r$，使得 $[l, r]$ 满足条件，那么我们可以用一个指针 $r$ 来**扩展**右端点。

如果 $r_i$ 存在单调性 $r_{i-1} \leq r_i$，那么**扩展**右端点总和至多 $O(n)$ 次。

## 5.1 分析

双指针是一种常见的**线性遍历**技巧，通常用于处理一些**区间问题**。

考虑固定左端点 $l$，我们可以用一个指针 $r$ 来**扩展**右端点。

如果我们记 $r_l$ 表示最小的 $r$，使得 $[l, r]$ 满足条件，那么我们可以用一个指针 $r$ 来**扩展**右端点。

如果 $r_i$ 存在单调性 $r_{i-1} \leq r_i$，那么**扩展**右端点总和至多 $O(n)$ 次。

其实即使不存在单调性，我们也可以用**二分**来处理。

此外，双指针的**扩展**不一定是线性移动的，包括有些时候**回撤**也不是。不过此时仍有可能有线性解决方案，这部分我们称之为**势能分析**。

## 5.2 例题

## 5.2 例题

### P1638 逛画展

博览馆正在展出由世上最佳的 $m$ 位画家所画的图画。

游客在购买门票时必须说明两个数字，$a$ 和 $b$，代表他要看展览中的第 $a$ 幅至第 $b$ 幅画（包含 $a, b$）之间的所有图画，而门票的价钱就是一张图画一元。

Sept 希望入场后可以看到所有名师的图画。当然，他想最小化购买门票的价格。

请求出他购买门票时应选择的 $a, b$，数据保证一定有解。

若存在多组解，输出 $a$ 最小的那组。

$1 \le n \le 10^6$，$1 \le a_i \le m \le 2 \times 10^3$。

## 5.2 例题

### P1638 逛画展

博览馆正在展出由世上最佳的 $m$ 位画家所画的图画。

游客在购买门票时必须说明两个数字，$a$ 和 $b$，代表他要看展览中的第 $a$ 幅至第 $b$ 幅画（包含 $a, b$）之间的所有图画，而门票的价钱就是一张图画一元。

Sept 希望入场后可以看到所有名师的图画。当然，他想最小化购买门票的价格。

请求出他购买门票时应选择的 $a, b$，数据保证一定有解。

若存在多组解，输出 $a$ 最小的那组。

$1 \le n \le 10^6$，$1 \le a_i \le m \le 2 \times 10^3$。

注意到 $r_i$ 单调不减，可以用数组维护**每幅画**有没有出现，再统计出现的画的种类，双指针即可，复杂度 $O(n)$。

## 5.2 例题

## 5.2 例题

### CF386C Diverse Substrings

给定一个字符串 $s$，定义 $d(x)$ 为字符串 $x$ 内不同的字符个数。

求有多少个 $s$ 的子串 $s'$，使得 $d(s') = i$，对 $i = 1 \sim d(s)$ 均求出答案。

$1 \le |s| \le 3 \times 10^5$。

## 5.2 例题

### CF386C Diverse Substrings

给定一个字符串 $s$，定义 $d(x)$ 为字符串 $x$ 内不同的字符个数。

求有多少个 $s$ 的子串 $s'$，使得 $d(s') = i$，对 $i = 1 \sim d(s)$ 均求出答案。

$1 \le |s| \le 3 \times 10^5$。

观察到 $d(s) \le 26$，那我们可以枚举 $i$，找到最近/最远的 $r_l, r'_l$，那么 $r \in [r_l, r'_l]$ 的字串 $s[l, r]$ 就都有 $d(s[l, r]) = i$。双指针即可，复杂度 $O(n)$。

## 5.3 习题

- **CF1771E Hossam and a Letter**
- **CF2084E Blossom**
- **P8434「WHOI-2」D&D**